top КОЛЛЕДЖ

Командная оболочка Bash (часть 4)

Написание скриптов

Операционные системы и среды. Практика

top КОЛЛЕДЖ

Создание командных файлов

Файлы скриптов .sh

Запуск и шебанг

Командные файлы

top КОЛЛЕДЖ

- Итак, командная оболочка Bash поддерживает исполнение командных файлов.
- ***Командный файл*** (или ***скрипт***, англ. ***shell script***) — это текстовый документ, который содержит набор команд оболочки (англ. ***shell***), выполняющихся друг за другом в собственном окружении в отдельном процессе.
* Традиционно в Unix для этих целей используются файлы с расширением .sh, однако фактически нет никакой разницы, какое у файла будет расширение. Часто можно увидеть скрипты без расширения вовсе (например, тот же самый начальный процесс init является скриптом).
* Командные файлы часто используются для автоматизации рутинных процессов, например, старта или инициализации служб (демонов).

Командные файлы

top КОЛЛЕДЖ

- Командный файл должен как минимум обладать правами на чтение, чтобы его можно было запустить в командной оболочке. Однако часто командному файлу выдается разрешение на исполнение.

☼Создайте файл script.sh и назначьте ему права на исполнение.

☼Для начала, сделаем очень простой скрипт, который будет что-нибудь писать в стандартный поток вывода. Для этого откройте файл в Vim и напишите в него несколько команд echo с произвольным текстом, разделенных переносом строки.

☼Выполните файл.

Исполнение скриптов

top КОЛЛЕДЖ

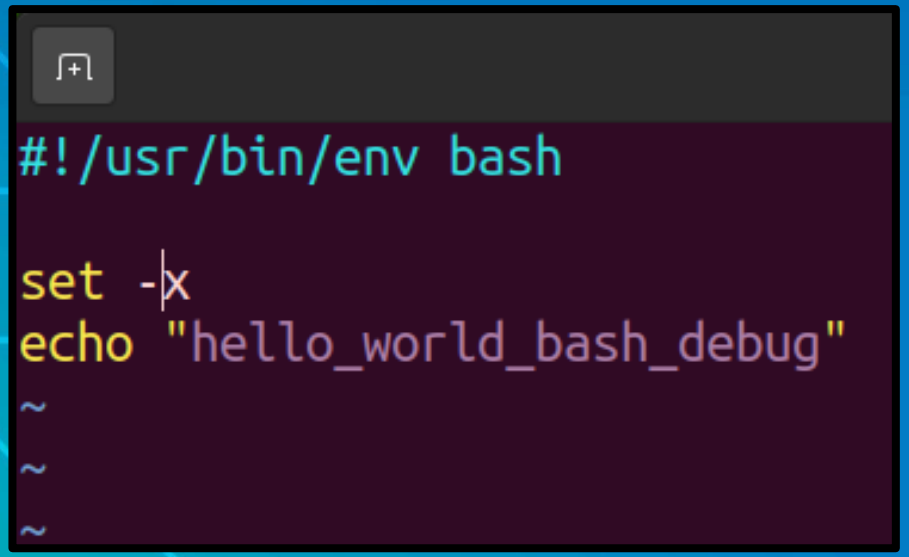
- Если у вас не получилось с первого раза запустить скрипт, как обычно запускаются другие команды, то не переживайте, этому есть объяснение.
- Дело в том, что командная оболочка при вводе некоторого слова, не являющегося абсолютным или относительным путем, пытается найти его среди заданных системных путей. Данные пути определены в переменной окружения PATH, однако об этом мы поговорим немного позже. Среди таких путей выделяются, как правило, `/bin`, `/usr/bin` и `/usr/sbin`.
- Так как текущий рабочий каталог (скорее всего) не указан в системных путях, то для выполнения скрипта из этого каталога необходимо указывать именно путь.

Шебанг

top КОЛЛЕДЖ

- Небольшое внимание стоит уделить такому понятию как «***шебанг***» (англ. ***sha-bang***).
- Шебангом в Unix-системах называют первую строку командного файла, начинающейся с последовательности символов #!, которая указывает, по какому абсолютному пути нужно использовать командный интерпретатор для данного командного файла. Например,

```
#!/bin/bash
```



Шебанг

top КОЛЛЕДЖ

* Мы говорим о командной оболочке Bash, однако она далеко не единственная. Существуют оболочки `sh`, `ash`, `zsh`, `csh` и некоторые другие.
* Более того, в шебанге может быть указан интерпретатор, не являющийся командной оболочкой, например,

```
#!/usr/bin/python
```

или даже

```
#!/home/user/random_program
```

если `random_program` реализует интерфейс командного интерпретатора.

- Укажите в шебанге вашего скрипта оболочку `/bin/bash`.
- Запустите скрипт с использованием относительного пути (*напоминание*: текущий каталог обозначается точкой).

top КОЛЛЕДЖ

Переменные и подстановки

Сохранение, получение, передача и использование переменных

Взаимодействие с переменными

Вызов команды в команде

Переменные

top КОЛЛЕДЖ

- Как и любой язык программирования, командная оболочка также поддерживает переменные.
- Создать переменную достаточно просто, для этого необходимо указать ее имя в самом начале строки и через равно без пробелов указать значение, например

```
my_str="Hello, world!"
```

- Bash по сути не имеет типов данных (за исключением некоторых структур), но любая переменная ближе всего по взаимодействию похожа на строку. Если в переменной нет пробелов, то ее можно записывать без кавычек, либо также можно экранировать пробелы обратной косой чертой.

Переменные

top КОЛЛЕДЖ

* Использование переменных осуществляется с помощью оператора доллар $. Этот оператор устанавливается перед именем переменной:

```
$my_str
```

- При использовании переменной в команде на ее место подставляется ее значение непосредственно перед вызовом. Например, команда

```
echo $my_str
```

будет эквивалентна

```
echo “Hello, world!”
```

что приведет к выводу в консоль значения переменной.

Чтение переменных

top КОЛЛЕДЖ

* Для получения значения из стандартного потока ввода и записи его в *переменную* используется команда

```
read [опции] [имена_переменных]...
```

- Если не указано ни одного имени переменной, то результат будет записан в переменную с именем REPLY.

☼ С помощью справки о команде read напишите скрипт, который будет запрашивать пароль, аналогично выполнению команды sudo, и затем выводить его в новой строке. *Пояснение*: скрипт должен написать сообщение об ожидании пароля, считать пароль без отображения вводимых символов и затем отобразить его на новой строке.

Подстановки

top КОЛЛЕДЖ

- Более общим случаем раскрытия переменной является ***подстановка*** (англ. ***substitution***, в документации также применяется термин ***expansion***).
- Bash обладает несколькими видами подстановок. Та, с которой мы познакомимся сейчас полностью называется подстановка параметра (англ. ***parameter expansion***).
- Подстановка параметра имеет вид (в данном случае фигурные скобки нужны!)

```
${[операция]переменная[операция]}
```

- При отсутствии выполняемой *операции* запись ${переменная} эквивалентна $переменная. Подстановка параметра является более универсальной по сравнению с раскрытием переменной.

Подстановка параметра

top КОЛЛЕДЖ

- Самые частые операции с подстановкой параметра (используется строка var):
  - `${var:N:M}` — взятие подстроки от *N*-ого до *M*-ого символа.
  - `${var/N/M}` — замена подстроки *N* на подстроку *M*.
  - `${var#N}` и `${var##N}` — удаление префикса, удовлетворяющего *N*.
  - `${var%N}` и `${var%%N}` — удаление постфикса, удовлетворяющего *N*.
  - `${#var}` — длина строки.
  - `${var:-N}` — подстановка *N*, если var не определено или пусто.
  - `${var^^N}` и `${var,,N}` — замена символов из *N* на верхний/нижний регистр.

Подстановка команды

top КОЛЛЕДЖ

* Еще одна возможность Bash — использовать вместо подстановки переменной ***подстановку команды*** (англ. ***command substitution***). Она позволяет выполнить команду, и ее результат сразу подставить в другую команду.
- Для этого используется синтаксис, похожий на обычную подстановку, но вместо фигурных скобок используются круглые:

```
$(команда)
```

- Пример использования:

```
echo “Process #1 is $(ps -p 1 -o comm --no-headers)”
```

* Заметьте, что подстановки можно записывать в строках, обособленных двойными кавычками. Строки же в одинарных кавычках выводятся как есть.

Подстановка команды

top КОЛЛЕДЖ

* Подстановку команды можно выполнять другой эквивалентной синтаксической конструкцией — с помощью обратных кавычек.

`команда`

- Пример использования:

```
echo “Process #1 is `ps -p 1 -o comm --no-headers`”
```

- Еще один частный случай подстановки команды — это подстановка процесса (англ. ***process substitution***), который позволяет результат одной команды сохранить в именованный канал и передать его в другой процесс:

>(команда) <(команда)

Арифметическая подстановка

top КОЛЛЕДЖ

* Если в переменной записано число, то с ним можно производить простые арифметические операции. Это может быть выполнено двумя способами:

1. Использование команды let:

```
let {выражение}
```

2. Использование ***арифметической подстановки*** (англ. ***arithmetic expansion***):

```
$((выражение))
```

- Недостаток использования `let` заключается в необходимости создания отдельной переменной для результата, в то время как арифметическая подстановка позволяет результат подставить в место расчета.

Получение аргументов

top КОЛЛЕДЖ

- У оператора доллар $ есть еще несколько применений. Первое — это получение аргументов командной строки.
- Если скрипт запускается с аргументами, то есть

```
./script.sh arg1 arg2 …
```

то можно получить аргументы с использованием оператора доллар и индекса аргумента:

```
$0, $1, $2, …
```

где $0 будет соответствовать строке ./script.sh, $1 — arg1, $2 — arg2 и так далее.

- Второе: количество аргументов можно получить с помощью $#.

Другие использования

top КОЛЛЕДЖ

- Третье: $* и $@ позволяет получить целиком всю командную строку, с помощью которой был запущен скрипт. Эквивалентно “$1 $2 $3 …”.
- Четвертое: $$ позволяет получить номер процесса, с которым запущен скрипт.
- Пятое: $RANDOM позволяет сгенерировать случайное число в пределах от 0 до 32567.
- Шестое, но наиболее важное: $? позволяет получить ***код возврата***, с которым завершилась последняя команда. Код возврата необходим для получения информации о том, как завершилось приложение: с ошибкой или успешно. С помощью этого можно управлять потоком команд скрипта.

Задания по подстановкам

top КОЛЛЕДЖ

- В последующих заданиях вы будете использовать ***подстановку параметра***.

☼ Напишите скрипт, который будет читать из стандартного потока предложение и заменять в нем все вхождения буквы `'a'` на собаку @. Необходимо вывести на экран первые десять и последние десять букв получившегося предложения.

☼ Напишите аналог команды `xargs`. Данная команда в Unix делает следующее:

- ☼ В команду передаются в виде аргументов другая команда с набором аргументов.
- ☼ В стандартный поток вывода отправляется строка, которая содержит другой набор аргументов.
- ☼ Первый набор аргументов вставляется в начале, второй — в конце общей команды, команда целиком выполняется.

- Пример: `echo "/etc/network/interfaces" | xargs grep "auto"`, эквивалентно команде `grep "auto" "/etc/network/interfaces"`.

Задания по подстановкам

top КОЛЛЕДЖ

- В последующих заданиях вы будете использовать ***подстановку команды***.

☼ Напишите скрипт, который будет выводить на экран количество файлов в указанной в аргументе директории. Для реализации используйте команды `ls` (список файлов) и `wc` (подсчет количества слов).

☼ Создайте скрипт, который будет собирать информацию в файл:

- ☼ В текущей директории должен быть создан файл с названием `info.txt`. По окончанию работы скрипта данный файл должен иметь права на чтение всеми пользователями и не должен иметь прав на запись.
- ☼ В файл необходимо записать информацию об имени пользователя, номере процесса, количестве файлов в домашней директории, процессе, потребляющем больше всего памяти, и времени запуска скрипта (воспользуйтесь командой `date`).

Задания по подстановкам

top КОЛЛЕДЖ

- В следующих заданиях вы будете использовать ***арифметическую подстановку.***

☼ Напишите скрипт, который будет вычислять и выводить объем четырехгранной пирамиды. Стороны a, b основания пирамиды и ее высота h передаются через аргументы в виде **целых неотрицательных** чисел. Ответ достаточно выдать в виде целого числа, проверять аргументы не нужно. Формула:

$$V = \frac{1}{3}abh$$

☼ Реализуйте калькулятор, который способен вычислить простое выражение с двумя вещественными числами до трех знаков после запятой. Вещественные числа должны разделяться запятыми или точками и передаваться в виде аргументов. Оператор тоже передается в виде аргумента.

top КОЛЛЕДЖ

Логические ветвления

Логические ветвления if/elif/else

Команда test, [, операторы && и ||

Команда if

top КОЛЛЕДЖ

- Bash поддерживает логические ветвления с использованием команды `if`. Синтаксис команды:

```
if {команда}
then
    …
fi
```

* Заметьте, что после `if` ставится не выражение, а именно команда. Если команда выполняется успешно (то есть ее код возврата равен нулю), то код в `if` также выполняется, иначе — пропускается до конца блока, ограниченного перевернутым `if` — `fi`.

Команда if

top КОЛЛЕДЖ

- Совместно с `if` могут использоваться операторы `elif` (для проверки еще одной команды) и `else` (для выполнения кода во всех остальных случаях):

```
if {команда}
then
    …
elif {команда}
then
    …
else
    …
fi
```

Проверка выражения

top КОЛЛЕДЖ

- В любых POSIX-совместимых командных оболочках существует специальная команда test, которая позволяет проверять выражения.

```
test {выражение}
```

* Если выражение истинно, то команда вернет нулевой код возврата, иначе — единицу. Для данной команды существует более удобный псевдоним:

```
[ {выражение} ]
```

Обратите внимание на **необходимость пробелов между выражением и квадратными скобками**.

- Подобные проверки часто используется в `if` на месте команды.

Проверка выражения

top КОЛЛЕДЖ

- На месте выражения может быть следующие проверки:

```
[ “$STR1” = “$STR2” ]        — равенство строк
[ “$STR1” != “$STR2” ]       — неравенство строк
[ -z “$STR” ]                — строка пустая
[ ! {EXPR} ]                 — отрицание выражения
[ -e {FILE} ]                — файл существует
[ $INT1 -{op} $INT2 ]        — сравнение чисел
```

- Здесь op может быть: eq — равно, ne — не равно, lt — меньше, gt — больше, le — меньше или равно, ge — больше или равно.

Задания с условиями

top КОЛЛЕДЖ

- Напишите скрипт, который будет заменять расширение файла, переданного в качестве аргумента в виде пути, на `.cpp`. Предусмотрите следующие действия: отсутствие переданного аргумента, отсутствие такого файла, отсутствие расширения файла, расширение файла уже равно `.cpp`.
- Напишите скрипт, который будет принимать на вход дату в виде трех численных аргументов (число, месяц, год) и проверять ее на корректность. Проверки включают количество дней в месяцах и високосные года.

Проверка выражения Bash

top КОЛЛЕДЖ

* У команды `test` есть небольшой недостаток: так как она работает «в лоб» (ведь по сути является отдельным исполняемым файлом), то все переменные нужно записывать в кавычках, иначе при работе с пустыми значениями на месте переменных будет возникать синтаксическая ошибка:

```
[ $EMPTY = “Not empty” ]  ⇨  [  = “Not empty” ]
```

- Поэтому вместо команды `test` или `[ … ]` исключительно в командной оболочке Bash можно использовать похожую конструкцию, лишенную этого недостатка:

```
[[ {выражение} ]]
```

которая также обладает большими возможностями.

Логические операторы

top КОЛЛЕДЖ

- В Bash существуют логические операторы && (И) и || (ИЛИ), характерные для обыкновенных языков программирования. Однако данные операторы работают не с логическими выражениями, а с кодами возврата команд.
- С помощью этих операторов можно объединять команды в цепочку, однако она следует следующим принципам (аналогично для языка Си):
  1. Если команда выполняется с ошибкой и за ней следует оператор &&, то команда после && не выполняется.
  2. Если команда выполняется успешно и за ней следует оператор ||, то команда после || не выполняется.
- Результирующий код возврата всей цепочки соответствует коду возврата последней успешно выполненной команды.

Запись команд в одну строку

top КОЛЛЕДЖ

- В скриптах Bash необязательно все команды разделять переносом строки, чтобы они выполнялись последовательно.
- Если необходимо записать команды в одну строку, и их поведение влияло на выполнение следующих команд, то можно использовать операторы && и ||.
- Если необходимо записать команды в одну строку, и они выполнялись принудительно вне зависимости от результата предыдущих команд, их можно разделять точкой с запятой «;».
- Сравните:

```
[ -e test.txt ] || exit 1
[ -e test.txt ]; exit 1
```

Конструкция case

top КОЛЛЕДЖ

- Если результат выполнения команды в условии if варьируется, то может пригодиться конструкция `case` вместо множества блоков `if-elif-else`. Данная конструкция проверяет значение на следование определенному шаблону:

```
case {значение} in
    {шаблон1})
        …
    ;;
    {шаблон2})
        …
    ;;
esac
```

top КОЛЛЕДЖ

Циклы

Цикл for и while

Циклы while и until

top КОЛЛЕДЖ

- В Bash есть несколько вариантов циклов, самые базовые — это while и until:

```
while {команда}
do
    …
done
```

```
until {команда}
do
    …
done
```

* Данные циклы взаимозаменяемые, так как `while` выполняет блок кода после do до тех пор, пока команда в условии выполняется успешно, а `until` — пока команда в условии выполняется с ошибкой.

Цикл for

top КОЛЛЕДЖ

- Второй цикл — это цикл for. В отличие от циклов while и until, он продолжает выполнять блок инструкций по заранее определенному *набору слов* (через пробел), при этом на каждой итерации следующее слово перезаписывается в переменную с заданным *именем*:

```
for {имя} in {набор}
do
    …
done
```

* Заметьте, что используется именно имя переменной, а не сама переменная, то есть `var`, а не `$var`.

Цикл for

top КОЛЛЕДЖ

- На месте набора слов может быть не только само перечисление, но и любая подстановка, а также *диапазон*, записывающийся как

{начало..конец}

```
for i in A B C D
do
    echo $i
done
```

```
for i in $(pgrep systemd)
do
    echo $i
done
```

```
for i in {1..5}
do
    echo $i
done
```

☼ Каков результат выполнения перечисленных сверху разных блоков кода?

Задание с циклами

top КОЛЛЕДЖ

- Напишите скрипт, который будет каждые несколько секунд писать в консоль сообщение. Для реализации используйте команду `sleep`.
- Напишите скрипт, который нарисует треугольник Паскаля заданной в первом аргументе глубины *N*.
- Напишите скрипт, который будет принимать на вход пароль и проверять его на соответствие критериям сложности: количество символов, наличие букв разного регистра и цифр, использование специальных символов. Также пароль не должен быть популярным (password, 11111111 и тому подобные).

Цикл for в стиле Си

- В Bash есть форма цикла for в стиле Си. В языке Си цикл for имеет три составляющих: первая часть указывает команду, выполняющуюся перед входом в цикл, вторая часть — условие продолжения цикла, третья часть — команду, выполняющуюся после каждой итерации цикла.
- В Bash данный цикл использует арифметические подстановки в качестве всех трех частей и имеет вид

```
for (( {1} ; {2} ; {3} ))
do
        …
done
```

Еще задания с циклами

top КОЛЛЕДЖ

- Создайте игру «Угадайка»:
  - В начале запуска скрипта генерируется случайное число от 0 до 10000.
  - Пользователь вводит случайное число, и программа определяет «больше», «меньше» или «равно».
- Создайте игру «Проверь реакцию»:
  - Каждый запуск сопровождается вопросом «`Ready?`»
  - Если пользователь нажимает Enter, то игра пишет сообщение «`Press ENTER on signal...`» и ждет случайное количество секунд (воспользуйтесь командой `sleep`).
  - После ожидания выводится сообщение «`NOW!`» и фиксируется текущее время (воспользуйтесь командой `date` и справкой по ней).
  - После нажатия Enter пользователем показывается разница между зафиксированным временем и нажатием в миллисекундах.

top КОЛЛЕДЖ

Функции

Вызов, передача аргументов и возврат значений

Функции

top КОЛЛЕДЖ

- Последнее, о чем стоит рассказать — это функции. ***Функцией*** называется блок кода с заданной сигнатурой (*именем*), который может быть использован многократно путем вызова из другого места программы.
- В Bash функции определяются следующей конструкцией:

```
{имя}()
{
        …
}
```

* Заметьте, что в отличие от языков группы Си, в Bash в скобках в определении функции не указываются аргументы.

Функции

top КОЛЛЕДЖ

* Вызов функции осуществляется по имени без указания скобок. При необходимости через пробел перечисляются аргументы:

```
{имя} [аргументы]…
```

* Обращение к аргументам осуществляется аналогично аргументам командной строки ($1, $2, …).
- В функции можно определять локальные переменные с помощью команды local:

```
local {имя}[=значение]
```

- Код возврата из функции устанавливается командой return:

```
return 0
```



Вопрос

top КОЛЛЕДЖ

Как вы думаете, в чем отличие использования команд exit и return?

Задание на функции и скрипты

top КОЛЛЕДЖ

- Напишите функцию, которая будет выводить текущую информацию о системе (например, рабочая директория, текущий пользователь и т.д.).
- Создайте функцию, которая выводит значение переменной по ее названию. Для реализации используйте команду `eval`.
- Напишите скрипт, который будет создавать резервную копию каталога, переданного по пути. В качестве результата должен быть сформирован архив с содержимым каталога и названием **“backup_{время}.tar”**. Для реализации воспользуйтесь утилитой tar и командой date.
- Напишите скрипт-игру для проверки реакции. Как только в терминале появляется некоторое сообщение, пользователь должен нажать Enter. По окончании выводится время в миллисекундах, потраченное на нажатие. Для реализации таймера используйте утилиту date.

top КОЛЛЕДЖ

Другие возможности

Массивы и передача переменных

Массивы

top КОЛЛЕДЖ

- В Bash существует встроенная возможность использования массивов. Массивы определяются как переменные с указанием *имени* и нескольких значений в круглых скобках через пробел.

```
{имя}=([значения]…)
```

- Для получения значения нужно использовать подстановку с указанием *индекса* в квадратных скобках:

```
${имя[индекс]}
```

- Можно получить сразу все значения из списка с помощью @ и * в качестве индекса:

```
${имя[@]}        ${имя[*]}
```

Передача переменных

top КОЛЛЕДЖ

- Если в качестве команд используются сложные утилиты или другие скрипты на Bash, то в них можно заранее передать переменные.
- ***Первый вариант*** — передача переменных по *имени* только для вызываемой утилиты может быть выполнена с помощью следующего синтаксиса:

  {имя}={значение}… {команда}
- ***Второй вариант*** — передача переменных по *имени* во все вызываемые утилиты (дочерние процессы) может быть выполнена с помощью команды export:

  export {имя}
- Примеры:

```
IFS=| echo “${LIST[@]}”
```

```
export var=1; ./script.sh
```

Краш-тест

top КОЛЛЕДЖ

- В Bash настолько «приятный глазу» синтаксис, что некоторые команды в нем вызывают удивление.
- Например, достаточно известный пример команды для «падения» системы:

```
:(){ :|:& };:
```

* Данная команда переопределяет `:`, которая по умолчанию является пустой командой, которая ничего не делает, на функцию, вызывающую саму себя рекурсивно с открытием нового процесса в фоне с этой же запущенной функцией.
* Другими словами, данная команда начинает бесконтрольно создавать процессы в ОС, полностью забивая стек и оперативную память.



Вопрос

top КОЛЛЕДЖ

Вопросы?